arXiv:2304.08793v2 [quant-ph] 18 Sep 2023

Quantum Architecture Search for Quantum Monte Carlo Integration via Conditional Parameterized Circuits with Application to Finance

Mark-Oliver Wolf
Department of Financial Mathematics
Fraunhofer ITWM
Kaiserslautern, Germany
mark-oliver.wolf@itwm.fraunhofer.de
0000-0002-3698-9266

Tom Ewen
Department of Financial Mathematics
Fraunhofer ITWM
Kaiserslautern, Germany
tom.ewen@itwm.fraunhofer.de

Ivica Turkalj
Department of Financial Mathematics
Fraunhofer ITWM
Kaiserslautern, Germany
ivica.turkalj@itwm.fraunhofer.de

***Abstract*—Classical Monte Carlo algorithms can theoretically be sped up on a quantum computer by employing amplitude estimation (AE). To realize this, an efficient implementation of state-dependent functions is crucial. We develop a straightforward approach based on pretraining parameterized quantum circuits, and show how they can be transformed into their conditional variant, making them usable as a subroutine in an AE algorithm. To identify a suitable circuit, we propose a genetic optimization approach that combines variable ansatzes and data encoding. We apply our algorithm to the problem of pricing financial derivatives. At the expense of a costly pretraining process, this results in a quantum circuit implementing the derivatives' payoff function more efficiently than previously existing quantum algorithms. In particular, we compare the performance for European vanilla and basket options.**

***Index Terms*—quantum computing, quantum finance, quantum monte carlo integration, quantum machine learning, quantum amplitude estimation**

I. Introduction

The potential for revenue and market disruption of quantum computing is enormous for the financial sector, with estimates going as high as $5 billion [1]. Showing a practical quantum advantage is crucial to attain these estimates. In gate-based quantum computing, we are currently in the era of Noisy Intermediate Scale-Quantum (NISQ) devices [2]. These are quantum devices with high error rates, therefore requiring potential quantum circuits to be extremely efficient in their resources such that the needed error mitigation schemes may be applied [3].

In finance, the pricing and risk management of assets, such as stocks, bonds, commodities, currencies and their derivatives is used for a variety of applications and regulations. Depending on the studied instrument, computing the necessary values on classical computers is complex and time consuming. There have been a variety of publications studying how the application of quantum computers may speed this up tremendously [4], [5]. A promising approach is the substitution of integration using classical Monte Carlo algorithms with its quantum counterpart, generally referred to as quantum Monte Carlo integration (QMCI) [6]. A variety of QMCI methods use a quantum algorithm called Amplitude Estimation (AE) [7], leading to a quadratic speed-up of the rate of convergence in the number of simulations [8], [9]. One crucial aspect of these simulations is the implementation of the respective functional dependencies on the quantum device [10]. Similar to classical simulations, these functions have to be applied multiple times in the estimation algorithm.

In this work, we present a novel method to substantially reduce the required quantum resources for this function implementation, at the cost of a computationally expensive pretraining on a classical device. Employing methods from the thriving area of quantum machine learning (QML) [11], we automatically generate the structure for a parameterized quantum circuit to implement payoffs of a financial derivative. Further, we introduce a technique based on substituting encoding circuits by controlled unitaries. We use the trained circuits with a varying number of qubits, which for example can be used for increasingly fine discretizations of the payoff function. This has two main advantages. First, one does not have to design a suitable circuit by hand, which, depending on the payoff, may not be trivial. Second, the resulting overhead for reusing a parameterized circuit scales only linearly with the number of qubits n used for the discretization, while implementing a function that can map 2^n different states to their respective values. Hence, easing the cost of simulations on a quantum computer. Interestingly, this approach evades the typical problem of data loading efficiency that is present in various QML applications [12]. Instead, we encode specific values as efficiently controlled operators. The result is a pretrained circuit that can be thought of as a conditional variant

This work was supported by the project AnQuC-3 of the Competence Center Quantum Computing Rhineland-Palatinate (Germany).

of the initial parameterized quantum circuit.

The article is structured as follows. The theoretical foundation of our algorithm is presented in Section II. We prove that the desired equality of expectations directly follows from an equality of individual encodings. This enables us to find an efficient conditional variant for circuits that use a specific rotational encoding. In Section III, we focus on finding a suitable, unconditonal circuit which we may later use to construct its conditional variant. Thereby, we present a genetic approach to automated circuit design, using prior results from Variable Ansatzes (VAns) [13] present in Variational Quantum Eigensolvers (VQE). This genetic approach is tailored for current NISQ devices and can be set to specifically use hardware-native quantum gates. In Section IV, we apply and compare the conditional parameterized quantum circuit to current state-of-the-art quantum algorithms for the pricing of financial derivatives, namely European vanilla options, as well as basket options with a fixed and variable weight. Surprisingly, the variable weight basket option only encodes the weight a single time at the start of the circuit. We conclude with a short summary and prospective future research in Section V.

II. Conditional Parameterized Quantum Circuits

We give a short overview why AE has the potential for a speed up compared to classical Monte-Carlo methods. Generally, they are used to calculate a variety of statistics for a given random variable X, respectively functionals $f(X)$. In quantitative finance, we are often interested in the expectation of $f(X)$. Unfortunately, knowing the distribution of X and the function f does not lead to us necessarily knowing anything about the distribution of $f(X)$. Especially, $\mathbb{E}[f(X)]$ is in general not the same as $f(\mathbb{E}[X])$. For many applications, there is no closed formula known for this expectation, so we have to estimate it by

$$\mathbb{E}[f(X)] \approx \frac{1}{M}\sum_{i=1}^{M} f(X_i), \tag{1}$$

where M is the number of realizations X_i of X. This sample mean estimator on the right-hand side converges with a rate of $O(1/\sqrt{N})$ by the Central Limit Theorem. For AE, we have to find a unitary operator $\mathcal{A}$ that constructs a quantum state with

$$\mathcal{A}\left|0\right\rangle_c\left|0\right\rangle = a\left|\varphi_0\right\rangle_c\left|0\right\rangle + b\left|\varphi_1\right\rangle_c\left|1\right\rangle,$$

for which we can now estimate the needed probability $|b|^2$ with a convergence of $O(1/N)$ w.r.t. the applications of $\mathcal{A}$. In our notation, the index c indicates the corresponding qubit register. By discretizing the range of X, we can find an operator [9], [14] such that

$$|b|^2 \approx \sum_{\omega\in\Omega} p_\omega f(X_\omega) = \mathbb{E}[f(X)], \tag{2}$$

where Ω is a discretized state space of X with probabilities p_ω and $X_\omega = X(\omega)$. This results in a theoretical speed up compared to classical algorithms.

Equation (2) can be realized by loading a distribution, implemented by the unitary P, on a control (or state) register of n qubits $\left|0\right\rangle_c$ and applying a function f, implemented by the unitary F, using an additional target qubit $\left|0\right\rangle_t$ depending on this control register:

$$F\left(P\otimes I_1\right)\left|0\right\rangle_c\left|0\right\rangle_t = \left|\varphi_0\right\rangle_c\left|0\right\rangle_t + \sum_{k=0}^{2^n-1}\sqrt{p_k}\left|k\right\rangle_c\sqrt{f(k)}\left|1\right\rangle_t. \tag{3}$$

In our notation, $|k\rangle$ for $k\in\mathbb{N}$ always refers to the computational basis state of the respective binary representation, and $|\varphi\rangle$ without additional comments are arbitrary quantum states. It is easier to find a quantum circuit for P for sufficiently smooth distributions [15]. But in general, the problem is nontrivial and referred to as Quantum State Preparation [16].

However, implementing the unitary F depends on the task at hand, and might introduce significant circuit depth, even if the underlying distribution is simple enough. In the following, we present a novel approach to accomplish this with smaller circuits.

A. Parameterized Quantum Circuits

One way to grasp hard-to-implement functions is to use parameterized quantum circuits (PQC) as function approximators. It has been shown that a sufficiently regular function can be approximated with arbitrary accuracy by a certain type of PQC. These results are based on the representation of a PQC and its expected measurement result by truncated Fourier series [17], [18]. To achieve a good approximation quality, PQCs with high depth are needed, making it problematic to run them on NISQ devices. Therefore, we also consider genetic algorithms, which can be designed to result in shallower PQCs.

A widely used form of PQCs, which we also consider in this paper, is given by

$$U(\boldsymbol{x},\boldsymbol{\theta}) := \prod_{l=1}^{L} S_l(\boldsymbol{x})W_l(\boldsymbol{\theta}^l), \tag{4}$$

where $\boldsymbol{x} = (x_1,\ldots,x_{\mathcal{K}}) \in \mathbb{R}^{\mathcal{K}}$ is a feature vector and for each $l = 1,\ldots,L$ we have that $\boldsymbol{\theta}^l = (\theta_1,\ldots,\theta_{Q_l}) \in \mathbb{R}^{Q_l}$ is a vector of parameters and

$$S_l(x) := \prod_{k=1}^{\mathcal{K}} T_{l,k}\cdot e^{-ix_kH_{l,k}}, \quad W_l(\boldsymbol{\theta^l}) := \prod_{q=1}^{Q_l} V_{l,q}\cdot e^{-i\theta_qG_{l,q}} \tag{5}$$

are quantum circuits that we denote as encoding and parameterized blocks, respectively. Here, $T_{l,k}, V_{l,q}$ are unitary and $H_{l,k}, G_{l,q}$ are hermitian matrices. We use $\boldsymbol{\theta} := \boldsymbol{\theta}^1\cdots\boldsymbol{\theta}^L \in \mathbb{R}^Q$ to abbreviate the concatenation of all parameters, where $Q := Q_1+\ldots+Q_L$.

We define the quantum model for $U(\boldsymbol{x},\boldsymbol{\theta})$ to be the function

$$\begin{aligned} f_U : \mathbb{R}^{\mathcal{K}}\times\mathbb{R}^Q &\longrightarrow \mathbb{R},\\ (\boldsymbol{x},\boldsymbol{\theta}) &\longmapsto \left\langle 0\right| U(\boldsymbol{x},\boldsymbol{\theta})^\dagger\mathcal{M}U(\boldsymbol{x},\boldsymbol{\theta})\left|0\right\rangle, \end{aligned}$$

where $\mathcal{M}$ is some observable. In QML, a typical choice is the σ_Z observable on one or more qubits, which we use as well if nothing else is specified.

For a given set of feature vectors $\mathcal{X} = \{\boldsymbol{x}^1, \ldots, \boldsymbol{x}^J\} \subset \mathbb{R}^{\mathcal{K}}$ and corresponding labels $\mathcal{Y} = \{y_1, \ldots, y_J\} \subset \mathbb{R}$, the quantum model f_U is trained by minimizing over $\boldsymbol{\theta}$ while averaging the distance between the quantum model of the data point and the corresponding labels over $\mathcal{X}$,

$$\underset{\boldsymbol{\theta} \in \mathbb{R}^Q}{\operatorname{argmin}} \left| f_U(\boldsymbol{x}^j, \boldsymbol{\theta}) - y_j \right|^2 .$$

We refer to the whole process of constructing $U(\boldsymbol{x}, \boldsymbol{\theta})$ and finding a minimizing set of parameters as a Variational Quantum Algorithm (VQA) [19], [20].

B. Conditional Variant to Implement State-Dependency

We are now introducing the main theoretical concept of this article, the conditional parameterized quantum circuit (CPQC). As mentioned in the introduction, one benefit of the CPQC approach is that one can calculate the weighted average of a function (approximated by a PQC) by evaluating a circuit on a suitable superposition of computational basis states. Here, we describe the idea in a more general context, where different distributions are loaded into different, not necessarily same sized, registers simultaneously.

Let $\mathcal{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{2^n}\} \subset \mathbb{R}^{\mathcal{K}}$ be a data set. For each dimension k of the feature vector, we assume a quantum register with n_k qubits. It should be emphasized again that we do not need these registers to be the same size. This means that if they are used for implementing discretizations of random variables, the chosen resolutions are allowed to differ. These registers are indexed by $r_1, \ldots, r_{\mathcal{K}}$ and called control registers. We denote with $n = \sum_{k=1}^{\mathcal{K}} n_k$ the total number of qubits used for the (entire) control register. Associated with each control register is a distribution $\mathcal{P}_k$ with $|p_{k_1}|^2, \ldots, |p_{k_{2^{n_k}}}|^2, p_k \in \mathbb{C}$ denoting the probabilities of the corresponding basis states. Further, we assume there to be an additional target register consisting of m qubits, indexed by t.

Let U be a unitary acting on the target register and let $c = \{i_1, \ldots, i_k\} \subseteq \{1, \ldots, n\}$ be a subset indicating a choice of qubits. As usual [21], the controlled operation $\Lambda(c, U)$ is defined by its action on the computational basis,

$$\Lambda(c, U) \left| b_1 \ldots b_n \right\rangle_c \left| j \right\rangle_t := \left| b_1 \ldots b_n \right\rangle_c U^{b_{i_1} \cdots b_{i_k}} \left| j \right\rangle_t ,$$

where $b_1, \ldots, b_n \in \{0, 1\}$ and $j = 0, \ldots, 2^{m-1}$. In other words, the unitary U is applied exactly if all $b_i = 1$ for qubits $i \in c$, otherwise we apply the identity. We use the notation $\Lambda^{(r)}(c, U)$ for the control operation acting on $n + m$ qubits, to indicate that the control qubits c are taken from the register r and that $\Lambda^{(r)}(c, U)$ acts trivially on all remaining registers. For some index set D, we call a product of control operations,

$$\Lambda := \prod_{k=1}^{\mathcal{K}} \prod_{d \in D} \Lambda^{(r_k)}(c_d, U_d)$$

a control circuit.

We now define the main concept of this paper.

Definition 1. (Conditional PQC)
Let $U := U(x, \boldsymbol{\theta}) = \prod_{l=1}^{L} S_l(\boldsymbol{x}) W_l(\boldsymbol{\theta}^l)$ be a PQC. A conditional parameterized quantum circuit (CPQC) associated to U is a unitary of the form

$$C_n(U) := \prod_{l=1}^{L} \Lambda_l \cdot (I_n \otimes W_l(\boldsymbol{\theta}^l)),$$

where Λ_l is a control circuit for each $l = 1, \ldots, L$.

The idea is that every entry of a feature vector $\boldsymbol{x}$ enters the CPQC through its designated control register via basis encoding. To make this idea more precise, we presume a mapping $b : \mathbb{R} \longrightarrow \{0, 1\}^n$ that specifies a translation of entries of a feature vector into basis states.

For each $k = 1, \ldots, \mathcal{K}$, the distribution $\mathcal{P}_k$ is loaded into the control register r_k via a unitary P_k defined through the relation

$$P_k \left| 0 \right\rangle_{r_k} = \sum_{i=0}^{2^{n_k}-1} \sqrt{p_{k_i}} \left| b(\boldsymbol{x}_i^k) \right\rangle_{r_k} .$$

The unitary $P := P_1 \otimes \cdots \otimes P_{\mathcal{K}}$ can then be interpreted as loading the joint distribution of independent random variables into the whole n-qubit control register.

Similar to PQCs, we define a quantum model for a CPQC to be the function $f_{C_n(U),P}$ defined by

$$\boldsymbol{\theta} \longmapsto \langle 0 | \left(P \otimes I_m\right)^\dagger C_n(U)^\dagger \mathcal{M} C_n(U) (P \otimes I_m) \left| 0 \right\rangle .$$

The observable $\mathcal{M}$ is always chosen as $I_n \otimes M$, if M is the observable of the associated PQC.

Remark 2. (Advantages of CPQC)
The ensuing question is how to construct $C_n(U)$ such that

$$\sum_{i=0}^{2^n-1} p_i f_U(\boldsymbol{x}^i, \boldsymbol{\theta}) = f_{C_n(U),P}(\boldsymbol{\theta}), \tag{6}$$

where the coefficients p_i are the values of the probability mass function of the joint distribution. Equation (6) states that, using $C_n(U)$, the classically expensive approximation in (1) can be performed on a quantum computer by applying the same PQC U, without changing the ansatz when the amount of data points increases.

The following proposition gives a sufficient condition for the satisfiability of the above equality.

Proposition 3. *(Equality of encodings)*
Let U be a PQC and $C_n(U)$ a corresponding CPQC. Then (6) is satisfied, if

$$(I_n \otimes S_l(\boldsymbol{x})) \bigotimes_k \left| b(\boldsymbol{x}_k) \right\rangle_{r_k} \left| \varphi \right\rangle_t = \Lambda_l \bigotimes_k \left| b(\boldsymbol{x}_k) \right\rangle_{r_k} \left| \varphi \right\rangle_t$$

for all $\boldsymbol{x} \in \mathcal{X}$ and all $\left| \varphi \right\rangle_t$ in the space of the target register.

Observe that the circuit on the left-hand side depends on $\boldsymbol{x}$, whereas the circuit on the right-hand side does not. The condition in the proposition means that one replaces the encoding circuits $S_l(\boldsymbol{x})$ by control circuits Λ_l that receive their

input via the control register. Both encoding procedures are chosen to produce the same quantum state for the same computational basis states. Proposition 3 ensures that this equality of basis states translates to the case of the control register being in a superposition, without possible phase kickbacks leading to additional error terms due to the repeated controlling procedure. The proof is based on the linearity of the expected value and can be carried out using standard rules for the kronecker product. It is given in the appendix.

In the next section, we illustrate the above proposition by concrete examples.

C. Efficient CPQCs

For brevity, we only consider examples of one-dimensional features in this section, i.e., $\mathcal{K} = 1$.

Example 4.
The following example shows how it is always possible to construct a CPQC in a trivial way that reproduces the result of a given PQC.

Let $\mathcal{X} = \{x_1, \dots, x_{2^n}\}$ be our data. For each $x \in \mathcal{X}$, let $c(x) \subseteq \{1, \dots, n\}$ be the set of indices representing the positions of 1 in $b(x)$. Let $U(\boldsymbol{x}, \boldsymbol{\theta}) = \prod_{l=1}^{L} S_l(\boldsymbol{x}) W_l(\boldsymbol{\theta}^l)$ be a PQC. For each $l = 1, \dots, L$, we define a control circuit as follows:

$$\Lambda_l(n) := \prod_{x \in \mathcal{X}} \Lambda(c(x), S_l(x)).$$

This circuit fulfills the requirement of Proposition 3. Note however, that for arbitrary data $\mathcal{X}$ and encoding blocks $S_l(x)$ constructing $\Lambda_l(n)$ needs $\Omega(2^{n+1})$ elementary (CNOT and one-qubit one-parameter rotational) gates [22].

We see that a concept of efficiency is needed to find a distinction of *bad* and *good* control circuits Λ. Hence, we introduce the following definition.

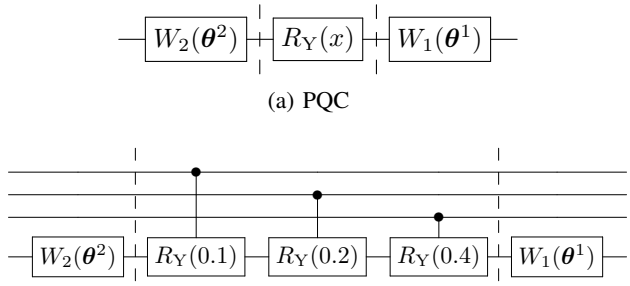
Definition 5. (Efficiency)
Let U be a PQC. Assume we have a procedure that generates a CPQC, $C_n(U)$, for every $n \in \mathbb{N}$. We call this construction efficient, if the minimum number of elementary gates needed to construct $C_n(U)$ scales linearly with n, i.e., is in $O(n)$.

We proceed with a small example showcasing an efficient CPQC construction, and afterwards generalize the concept for arbitrarily sized control registers and PQCs that have direct encodings via rotational gates.

Example 6.
We choose $n = 3$ control qubits and $m = 1$ target qubit. Let $\mathcal{X}$ and $b : \mathcal{X} \longrightarrow \{0,1\}^3$ be given by the following table:

Index i	0	1	2	3	4	5	6	7
$x_i \in \mathcal{X}$	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7
$b(x_i)$	000	001	010	011	100	101	110	111

The PQC consists of one layer only and the encoding circuit used is $S_1(x) = R_{\mathrm{Y}}(x)$. The parameterized circuits $W(\boldsymbol{\theta})$ and the control circuits Λ_l used to construct the CPQC are shown in Fig. 1. It is evident that, with the data and encoding chosen as above, the requirement of Proposition 3 is satisfied.



(a) PQC

(b) CPQC

Fig. 1: The circuit construction for Example 6. (a) The given PQC U with direct encoding of x as a Y-rotational gate. (b) The corresponding CPQC implementing the eight possible values of $\mathcal{X}$ depicted in the table as controlled Y rotations.

Example 7.
The previous example can be generalized to arbitrarily fine discretizations. Let $n \in \mathbb{N}$ be an arbitrary number of qubits used for the control register. Let our data $\mathcal{X} = \{x_1, \dots, x_{2^{n-1}}\}$ be given by the discretization $x_k = \frac{2\pi}{2^n} k$. Each x_k is encoded in the control register by the bit-representation of the integer k. Our PQC, $U(x, \boldsymbol{\theta}) = \prod_{l=1}^{L} S_l(x) W_l(\boldsymbol{\theta}^l)$, can have arbitrary many layers, with each layer having an encoding circuit of the form $S_l(x) = R_\alpha(x), \alpha \in \{\mathrm{X}, \mathrm{Y}, \mathrm{Z}\}$. The parameterzied blocks $W_l(\boldsymbol{\theta}^l)$ can be chosen arbitrarily. In this setup, each encoding circuit $S_l(x)$ can be substituted by a control circuit $\Lambda_l(n)$ acting on $n+1$ qubits,

$$\Lambda_l(n) = \prod_{c=0}^{n-1} \Lambda(c, R_\alpha),$$

such that the requirements of the Proposition 3 are satisfied. Note that the construction of $\Lambda_l(n)$ uses n factors; each factor can be constructed by two single qubit operations and two CNOTs. Thus, this construction scales like $O(n)$ and is efficient in the sense of Definition 5.

III. Implementation of Genetic PQCs

VQAs show a lot of potential for enabling noisy intermediate-scale quantum devices to solve practical tasks. As discussed in recent literature, choosing a suitable PQC is not a trivial task and has a huge influence on the performance of VQAs [23]. There are two important requirements while choosing the ansatz; it needs to be expressive enough to model the problem at hand and it needs to be as shallow as possible. Unfortunately, these two goals often contradict.

An idea, that recently came up in different variations, is algorithmically adapting the ansatz to the problem at hand. Some of the developed algorithms are ADAPT-VQE [24], Rotoselect and Rotosolve [25], Quantum Circuit Evolution of Augmenting Topologies (QCEAT) [26] and the already mentioned Variable Ansatzes (VAns) [13]. Especially under the term Quantum Architecture Search (QAS) some recent

research has been published [23], [27], [28]. These works mainly focus on the VQE. This means that the goal is to prepare one specific state, while we on the contrary want to learn functions. The additional difficulty is integrating the dependency on the input data, also called the encoding, into an adaptive scheme.

In this section we will extend on the ideas of VAns.

A. Variable Ansatz Algorithm

The general idea is to iteratively alternate between adding randomly selected parameterized or encoding blocks, as defined in (5), and removing noncontributing gates.

More formally the algorithm consists of:

- Training data consisting of inputs $\mathcal{X} = \{x_1, \ldots, x_J\}$ and corresponding true values $\mathcal{Y} = \{y_1, \ldots, y_J\}$. Usually the y_i are real numbers and the x_i are either real values or real vectors.
- An initial parameterized circuit $U_0(x, \theta_0)$, that depends on the input x and a parameter vector θ_0 of suitable size.
- A cost function $C(\mathcal{X}, \mathcal{Y}, U, \theta)$ that gives some measure of prediction error. For example, the mean squared error between the prediction and the true values.
- A classical optimizer Υ that minimizes the costs. Ideally $\Upsilon(\mathcal{X}, \mathcal{Y}, U) = \min_\theta C(\mathcal{X}, \mathcal{Y}, U, \theta)$.
- A set of blocks that we can add to the circuit.
- A decision function $g(c, \hat{c})$ that gives an acceptance probability given the old and the new costs.
- A reduction function $\rho(U)$ that removes noncontributing gates.
- The number of iterations n_i.

Algorithm 1 Structure Learning

$c_0 = \Upsilon(\mathcal{X}, \mathcal{Y}, U_0)$.
for $i = 1, \ldots, n_i$ **do**
  Sample block to add, qubits on which it operates, position in the circuit.
  Construct new circuit $\hat{U}_i$.
  Set $\hat{c}_i = \Upsilon(\mathcal{X}, \mathcal{Y}, \hat{U}_i)$.
  Sample Z uniformly distributed between 0 and 1.
  if $Z \leq g(c_{i-1}, \hat{c}_i)$ **then**
    $U_i = \rho(\hat{U}_i)$.
    $c_i = \Upsilon(\mathcal{X}, \mathcal{Y}, U_i)$.
  else
    $U_i = U_{i-1}$.
    $c_i = c_{i-1}$.
  end if
end for

A qualitative demonstration of how the circuit changes during one step of this algorithm is given in Fig. 2.

In the initial proposal for that algorithm [13], the set of possible blocks only consisted of parameterized unitaries $U(\theta)$, for which there exists a θ^* such that $U(\theta^*)$ is the identity. This ensures that adding these blocks to the circuit, while adding θ^* to the parameter vector, does not change the circuit and, thus, does not increase the cost. If then gradient descent is used to optimize the new set of parameters, there is a chance for a reduction of cost. In conclusion, the cost is, not necessarily strictly, decreasing in each step.

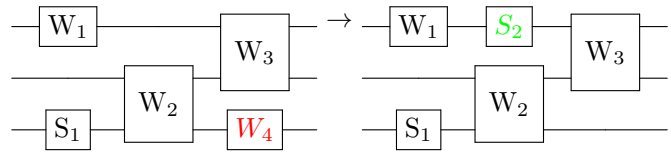


Fig. 2: Schematic visualization of one step of Algorithm 1, the block S_2 gets added while the block W_4 got removed.

For the parameter optimization of one PQC we use gradient based optimizers from the pennylane framework [29]. For the results of this work the gradients where calculated by automatic differentiation, which implies the use of noiseless simulators.

This algorithm allows for some modifications that will benefit the special requirements of our use case. There are three main points we can modify.

Blocks to add: While the original paper only focused on parameterized blocks that are used for training, we also allow encoding blocks. They violate the requirement to be the identity with the right parameters, but will greatly improve the capability of approximating arbitrary functions. For datasets with more than one input feature, the set of blocks to sample from contains individual encoding blocks for every input feature. This can also be interpreted as a form of feature selection.

Gate Removal: By allowing a very slight increase in the cost, we can often leave out many gates. As NISQ devices produce some error themselves, a slight increase in prediction error can be a good tradeoff for a simpler circuit. By further preventing some of the gates to be removed during the iterations, we try to keep the search space as big as possible. Examples for gates we do not remove are the last entanglement a qubit has with a measured one or the last parameterized gate on a qubit.

Hardware Awareness: This approach also allows us to take the hardware into account, which the circuit should be executed on. For example, we can only allow blocks of native gates and only entangle qubits that have a physical connection. Additionally we can adapt the algorithm to limitations of NISQ devices by modifying the cost function to also take the depth of the circuit and the number of CNOT gates into account.

B. Genetic Optimization

In this section we propose to adapt techniques from classical genetic optimization, in order to improve the optimization of the circuit structure laid out in Algorithm 1.

Genetic optimization keeps track of a population of possible solutions and applies three concepts on this population [30]:

- Mutation: Randomly modify one possible solution.
- Crossover: Combine different solutions to form a new one.

- Selection: Deciding which solutions will be used in the next generation.

In our approach, the crossover step is problematic, as it is not clear if and why the combination of two parameterized circuits should perform better than the individual ones. Thus, we focus on mutation and selection. In addition to the inputs from Algorithm 1, we need a number of generations n_g and the size of the population n_p.

Algorithm 2 Genetic Structure Learning

Initialize circuits $\mathbf{U}_0 = (U_0^j)_{j=1,\dots,n_p}$.
for i = 1,..., n_g **do**
  Mutation: Apply Algorithm 1 to all elements of $\mathbf{U}_{i-1}$ and store cost of each circuit c_{i-1}.
  Selection: Sample new generation $\mathbf{U}_i$ by sampling n_p times from $\mathbf{U}_{i-1}$ where the probability to choose U_{i-1}^j is given by $w_j = \frac{1}{c_j^2 \sum_{k=1}^{n_p} \frac{1}{c_k^2}}$.
end for
Choose the best circuit from $\mathbf{U}_{n_g}$ as output.

C. Structure Learning for CPQCs

For the application part we want to take circuits generated by Algorithms 1 and 2 and build controlled versions of them. Here we will demonstrate which choices we make in these algorithms to be able to do just that. At first, we require that the initial circuit is of the form (4). As this form is very general, this is not a real restriction. Further, we only allow blocks of the structure (5) to be added. In every step we only add one encoding or parameterized block, but this can still be treated as one layer, as the identity is a valid example for both kinds of blocks. We restrict the encoding blocks to simple rotations, i.e., the $T_{l,k}$ are chosen as the identity and $H_{l,k}$ is as one of $\sigma_x/2$, $\sigma_y/2$ or $\sigma_z/2$. At last, we have to make sure to not break the structure by removing individual gates in the reduction step. By increasing the number of layers, we can write every individual gate as one layer. Thus, removing a single gate, i.e., one layer, we keep the structure from (4). This enables us to use the encoding scheme described in Section II.

IV. Application to Option Pricing

We apply the presented methodology to the problem of derivative pricing in finance and compare our method to current state-of-the-art quantum algorithms. Assume that in general we have underlying securities X with some distribution $\mathcal{P}$ under a fitting probability measure $\mathbb{Q}$, and an option payoff $f(X)$. Assume $x_k \in [0, 2\pi)$ is a scaled discretization of X with $k \in \{0, \dots, 2^n - 1\}$, and $b(x_k)$ its bit representation. Recall, that we assume the operator P implementing the distribution to be given. We present an efficient implementation of the operator F to construct the needed quantum state (c.f., (3))

$$|\varphi_0\rangle_c |0\rangle_t + \sum_{k=0}^{2^n-1} \sqrt{p_k}\, |b(x_k)\rangle_c \sqrt{f(x_k)}\, |1\rangle_t \,,$$

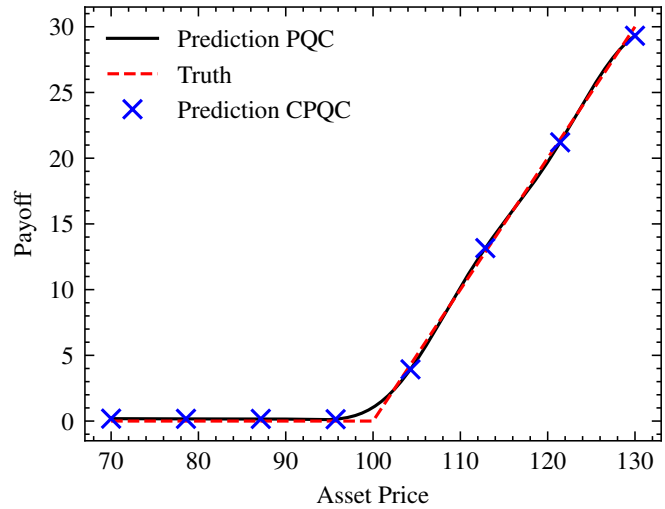


Fig. 3: Function approximation by a PQC that was found by Algorithm 2 and its CPQC version. The function approximated is the payoff of an European Call with a Strike of 100.

where $|\varphi_0\rangle$ is an arbitrary state.

A. European Vanilla Options

When pricing European vanilla options, we have a single underlying security $X = X^{(1)}$ with a fixed maturity T we are omitting in the notation for simplicity. Then, the payoff of a call option f_+ or put option f_- with a fixed strike K is defined as

$$f_\pm(X) := \max\left(\pm(X - K), 0\right).$$

The option price can be calculated by discounting the expected payoff [31]. We assume this discounting to be constant and applied in post-processing on a classical device. We denote with capital $X_k, k = 0, \dots, 2^n - 1$ the discretized version of X.

Quantum arithmetic: We compare our method to the implementation by IBM's qiskit finance module [32], based on the current state-of-the-art method introduced by References [9], [14]. They priced a vanilla option by using a quantum comparator circuit that flips an ancilla qubit initially in state $|0\rangle$ to the state $|1\rangle$ if $X_i > K$, using $O(n)$ gates. Afterwards, introducing a scaling factor $\tilde{c}$ they use the approximation $\sin^2(y + \pi/4) \approx y + 1/2$ for small $|y|$ to efficiently implement the $\pm(X - K)$ by linearly many Y-rotational gates, each controlled by only two qubits, on a second ancilla qubit. For the European call option, the resulting probability to measure this second ancilla qubit in state $|1\rangle$ is given by

$$\frac{1}{2} - \tilde{c} + \frac{2\tilde{c}}{\max_i X_i - K} \sum_{X_i \geq K} p_i (X_i - K),$$

where the option price can be recovered by rescaling. The entire quantum arithmetic implementation scales as $O(n)$ in the qubit register size n, i.e., in $O(\log(N))$ for $N = 2^n$ discretization points of the underlying.

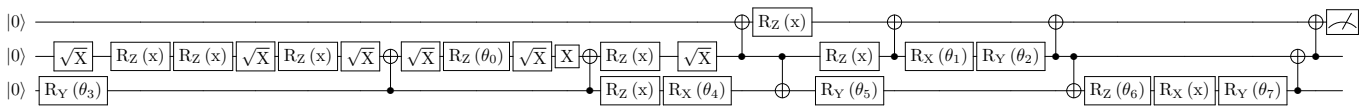


Fig. 4: The trained PQC for the payoff of a European call option generated by Algorithm 2. $\boldsymbol{\theta}$ = [0.93963, 2.51976, -0.30702, -0.22985, -0.302, -0.09293, 0.15291, 0.05979]

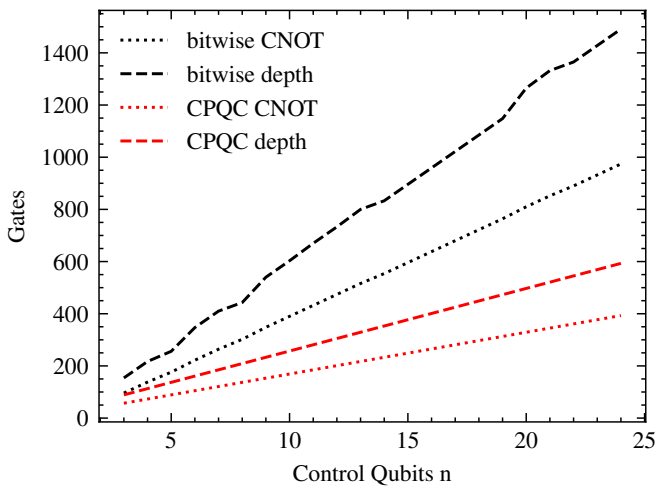


Fig. 5: Total CNOT count and depth comparison of the quantum arithmetic approach by IBM and our CPQC method explained in Section IV-A. Counts were computed with respect to the CNOT and $\mathrm{U}(\theta, \varphi, \lambda)$ gate decomposition.

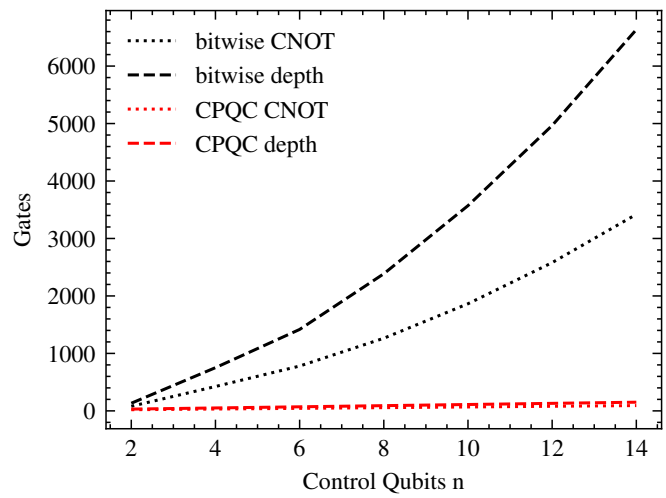


Fig. 6: Total CNOT count and depth comparison of the quantum arithmetic approach by IBM and our CPQC method explained in Section IV-B. We chose two underlying securities $\boldsymbol{X} = (X^{(1)}, X^{(2)})$, so $\mathcal{K} = 2$ and $n_k = n/2$. The weight was set to $\boldsymbol{w} = (2/3, 1/3)$, which was implemented for the quantum arithmetic algorithm for the scaled up integer values $w_1 = 2, w_2 = 1$. Counts were computed with respect to the CNOT and $\mathrm{U}(\theta, \varphi, \lambda)$ gate decomposition. For $n = 14$, the CPQC has a CNOT count of 92 and a depth of 151.

CPQC: We used Algorithm 2 to generate the circuit seen in Fig. 4, with the corresponding CPQC being shown in Fig. A.2. The resulting function approximation is demonstrated in Fig. 3. Since a PQC can be represented as a truncated Fourier series, where the frequencies included depend on the extent of data reuploading [17], we observe a smoothing of the learned function at the one non differentiable point. The genetic algorithm simulated 20 generations with a population size of 48. Each generation did 20 iterations of structure optimization. The parameter optimization was done by Pennylanes Root mean squared propagation optimizer. A new step was accepted if the costs decreased; whereas if the costs increased the probability to accept the new structure was calculated as $\exp(-5(c_{new} - c_{old})/c_{old})$. A gate was removed if the removal lead to an increase in cost below 1%. The whole procedure took about 5.5 hours on a dual Intel Xeon Gold 6240R system with 48 cores. We can see a comparison of the required number of CNOT gates and depth scaling of both decomposed quantum circuits in Fig. 5. The decomposition is done with regards to CNOT gates and $\mathrm{U}(\theta, \varphi, \lambda)$ gates.

B. Basket Option - Fixed Weight

A basket option is defined for a vector of underlying securities $\boldsymbol{X} = (X^{(1)}, ..., X^{(\mathcal{K})})^T$, fixed weights $\boldsymbol{w} = (w_1, \ldots, w_{\mathcal{K}}), w_k \in [0, 1], \sum_k w_k = 1$, and a fixed strike K. Its payoff profile is given by

$$f_{\text{Basket}}(\boldsymbol{X}) := \max\left(\boldsymbol{w}\boldsymbol{X} - K, 0\right).$$

Quantum arithmetic: Again, we compare our approach to the implementation by IBM qiskit based on Reference [9]. Each underlying $X^{(i)}$ is represented by its discretization on a separate qubit register r_i, in total needing n qubits. The circuit to implement the payoff is now similar to the vanilla option, except that an additional operator computing the weighted sum of the qubit registers is needed before applying the comparator. This sum is saved into a new set of ancilla qubits of size $\log_2(\sum_k w_k) + 1$. Then the methodology of the vanilla option is applied to this new register containing the weighted sum. Because of this weighted sum operator, the discretization for all securities has to be the same. It needs $O(n \log_2 n)$ gates, which is also the total complexity of the quantum arithmetic implementation of F.

CPQC: This task shows the main improvement of the CPQC approach. For a chosen level of approximation, the PQC

has a fixed qubit and gate number. Coupling the PQC with the control register via the method described in Example 7 only adds a linear amount of gates, resulting in a total gate complexity for the CPQC of $O(n)$. The genetically trained PQC and corresponding CPQC for this task are presented in Fig. A.3 and Fig. A.4, respectively. The PQC approximation, as well as the f_{Basket}, can be found in Fig. A.1.

With the same procedure used as for the call option, we can see a comparison of the required number of CNOT gates and depth scaling of both decomposed quantum circuits in Fig. 6.

C. Basket Option - Variable Weight

Although it substantially increases the difficulty for the quantum arithmetic implementation, it is straightforward to include a variable weight for the basket option as an input feature of our PQC, hence, of the CPQC. By doing so, one could pre-train a sophisticated PQC for a specific set of underlyings and a corresponding resolution. This, albeit possibly needing a larger circuit, has a resulting CPQC that has a gate scaling linear in the number of state qubits needed to represent the securities $X^{(i)}$. This CPQC can then either be used for a variety of different weights (in a range that was previously used for training), or for computing the prices of a basket where the weights depend on some different state register. The result of our genetically trained PQC, depicted in Fig. A.5, can be seen in Fig. 7. It is straightforward to construct the CPQC from this PQC as all the encoding blocks are rotational gates. Of course, the prediction could be improved by employing a larger PQC at the cost of more depth. At least for two underlyings, it should be emphasized that the variable weight w_1 is only encoded once in the PQC. It seems to rotate the payoff function, which the PQC seems to be able to implement well. These observations might also be useful for finding circuits, that implement different payoffs, *by hand*.

Additionally, this implementation of a basket option might open up the possibility for pricing index options that have a combination of stocks as the underlying. We leave this for future research.

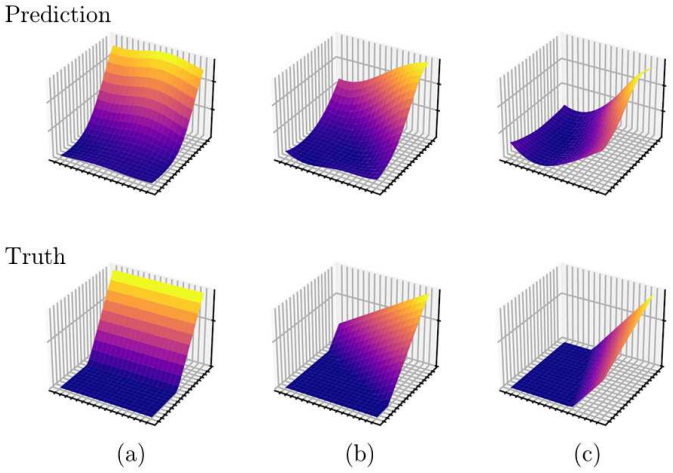


Fig. 7: Result of the circuit from Fig. A.5 in the upper row compared with the true payoff function f_{Basket} in the lower row. This circuit takes three inputs, the value of the two assets and the weight of the first asset. The figure shows the results three different weights: (a) $w_1 = 0$, (b) $w_1 \approx 0.37$, and (c) $w_1 \approx 0.74$.

V. Conclusion

We presented an automated approach to the function implementation for the quantum alternative of classical Monte Carlo algorithms based on quantum machine learning. First, laying the theoretical groundwork to build a conditional variant, we introduced our own genetic algorithm to find a suitable PQC. Second, we compared our approach to current quantum algorithms for the task of option pricing, which rely on a quantum version of bitwise computations. We were able to show that at the cost of a classical pre-training and a preemptively chosen approximation level, we generated reusable CPQCs that need a fraction of the quantum resources compared to the bitwise algorithms.

Possible future research is the further improvement of the genetic optimization of the PQC. One possible direction is taking the hardware into account by using only native gates, only entangling physically connected qubits, weighting approximation error against hardware noise and also optimizing for robustness in the presence of noise. Additionally one could adapt the algorithm to make sure that the inverse of the function operator F is also efficient, which would be a benefit for the application of AE. In the same vein, it should be researched if the application of PQCs translates to shortcuts one could take for quantum Monte Carlo integration algorithms. This has previously been done for other approaches, e.g., spin-echo circuit optimization [10]. The CPQC approach might also make existing AE techniques that have additional conditions on their function operator easier to implement [33]. Of course, a sophisticated ressource estimation for quantum advantage including the CPQC approach should be made as well, c.f., [34].

We are currently investigating the possibility of generalizing the conditional variant of PQCs for encodings other than simple rotations, which will probably translate to a slight encoding error, and more sophisticated control circuits Λ. Nevertheless, in the current NISQ environment small approximation errors perish in comparison to noise introduced by quantum gates. It has to be studied at which level of reduction in gate counts this additional error is justified.

References

[1] I. I. for Business Value. (2019) Exploring quantum computing use cases for financial services. [Online]. Available: https://www.ibm.com/downloads/cas/2YPRZPB3

[2] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 08 2018. [Online]. Available: https://doi.org/10.22331/q-2018-08-06-79

[3] C. Bauckhage, R. Bye, A. Iftikhar, C. Knopf, M. Mustafic, N. Piatkowski, R. Sifa, R. Stahl, and E. Sultanow, "Quantum Machine Learning - State of the Art and Future Directions," *Federal Office for Information Security https://www.bsi.bund.de*, p. 122, 2022.

[4] R. Orús, S. Mugel, and E. Lizaso, "Quantum computing for finance: Overview and prospects," *Reviews in Physics*, vol. 4, p. 100028, 11 2019. [Online]. Available: https://doi.org/10.1016/j.revip.2019.100028

[5] D. Herman, C. Googin, X. Liu, A. Galda, I. Safro, Y. Sun, M. Pistoia, and Y. Alexeev, "A Survey of Quantum Computing for Finance," 06 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2201.02773

[6] P. Intallura, G. Korpas, S. Chakraborty, V. Kungurtsev, and J. Marecek, "A Survey of Quantum Alternatives to Randomized Algorithms: Monte Carlo Integration and Beyond," 03 2023, arXiv:2303.04945 [quant-ph, stat]. [Online]. Available: http://arxiv.org/abs/2303.04945

[7] G. Brassard, P. Hoyer, M. Mosca, and A. Tapp, "Quantum Amplitude Amplification and Estimation," *arXiv:quant-ph/0005055*, 05 2000. [Online]. Available: https://doi.org/10.48550/arXiv.quant-ph/0005055

[8] P. Rebentrost, B. Gupt, and T. R. Bromley, "Quantum computational finance: Monte Carlo pricing of financial derivatives," *Phys. Rev. A*, vol. 98, no. 2, p. 022321, 08 2018. [Online]. Available: https://doi.org/10.1103/PhysRevA.98.022321

[9] N. Stamatopoulos, D. J. Egger, Y. Sun, C. Zoufal, R. Iten, N. Shen, and S. Woerner, "Option Pricing using Quantum Computers," *Quantum*, vol. 4, p. 291, 7 2020. [Online]. Available: https://doi.org/10.22331/q-2020-07-06-291

[10] A. C. Vazquez and S. Woerner, "Efficient State Preparation for Quantum Amplitude Estimation," 05 2020. [Online]. Available: https://arxiv.org/abs/2005.07711v1

[11] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 09 2017. [Online]. Available: https://doi.org/10.1038%2Fnature23474

[12] M. Schuld and F. Petruccione, *Machine Learning with Quantum Computers*, ser. Quantum Science and Technology. Springer International Publishing, 2021.

[13] M. Bilkis, M. Cerezo, G. Verdon, P. J. Coles, and L. Cincio. A semi-agnostic ansatz with variable structure for quantum machine learning. [Online]. Available: https://doi.org/10.48550/arXiv.2103.06712

[14] S. Woerner and D. J. Egger, "Quantum risk analysis," *npj Quantum Inf*, vol. 5, no. 1, feb 2019. [Online]. Available: https://doi.org/10.1038%2Fs41534-019-0130-6

[15] L. Grover and T. Rudolph, "Creating superpositions that correspond to efficiently integrable probability distributions," 2002. [Online]. Available: https://doi.org/10.48550/arXiv.quant-ph/0208112

[16] I. F. Araujo, D. K. Park, F. Petruccione, and A. J. da Silva, "A divide-and-conquer algorithm for quantum state preparation," *Scientific Reports*, vol. 11, no. 1, 03 2021. [Online]. Available: https://doi.org/10.1038%2Fs41598-021-85474-1

[17] M. Schuld, R. Sweke, and J. J. Meyer, "Effect of data encoding on the expressive power of variational quantum-machine-learning models," *Phys. Rev. A*, vol. 103, p. 032430, Mar 2021. [Online]. Available: https://doi.org/10.1103/PhysRevA.103.032430

[18] Z. Yu, H. Yao, M. Li, and X. Wang, "Power and limitations of single-qubit native quantum neural networks," 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2205.07848

[19] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, pp. 625 – 644, 2020. [Online]. Available: https://doi.org/10.1038/s42254-021-00348-9

[20] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications*, vol. 5, no. 1, 7 2014. [Online]. Available: https://doi.org/10.1038/ncomms5213

[21] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[22] M. Mottonen, J. J. Vartiainen, V. Bergholm, and M. M. Salomaa, "Transformation of quantum states using uniformly controlled rotations," 2004. [Online]. Available: https://doi.org/10.48550/arXiv.quant-ph/0407010

[23] Y. Du, T. Huang, S. You, M.-H. Hsieh, and D. Tao, "Quantum circuit architecture search for variational quantum algorithms," *npj Quantum Information*, vol. 8, no. 1, pp. 1–8, 05 2022. [Online]. Available: https://doi.org/10.1038/s41534-022-00570-y

[24] H. R. Grimsley, S. E. Economou, E. Barnes, and N. J. Mayhall, "An adaptive variational algorithm for exact molecular simulations on a quantum computer," *Nature Communications*, vol. 10, no. 1, p. 3007, 07 2019. [Online]. Available: https://doi.org/10.1038/s41467-019-10988-2

[25] M. Ostaszewski, E. Grant, and M. Benedetti, "Structure optimization for parameterized quantum circuits," *Quantum*, vol. 5, p. 391, 01 2021. [Online]. Available: https://doi.org/10.22331/q-2021-01-28-391

[26] Y. Huang, Q. Li, X. Hou, R. Wu, M.-H. Yung, A. Bayat, and X. Wang, "Robust resource-efficient quantum variational ansatz through evolutionary algorithm," *Physical Review A*, vol. 105, no. 5, p. 052414, 05 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2202.13714

[27] E.-J. Kuo, Y.-L. L. Fang, and S. Y.-C. Chen, "Quantum architecture search via deep reinforcement learning," 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2104.07715

[28] S.-X. Zhang, C.-Y. Hsieh, S. Zhang, and H. Yao, "Differentiable quantum architecture search," *Quantum Science and Technology*, vol. 7, no. 4, p. 045023, aug 2022. [Online]. Available: https://doi.org/10.1088%2F2058-9565%2Fac87cd

[29] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, and S. Ahmed et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. [Online]. Available: https://doi.org/10.48550/arXiv.1811.04968

[30] M. Melanie, *An Introduction to Genetic Algorithms*. The MIT Press, 03 1998.

[31] R. Korn, E. Korn, and G. Kroisandt, *Monte Carlo Methods and Models in Finance and Insurance*, M. Dempster, D. B. Madan, and R. Cont, Eds. Boca Raton, FL: CRC Press, 2010.

[32] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2023. [Online]. Available: https://doi.org/10.5281/zenodo.2573505

[33] K. Plekhanov, M. Rosenkranz, M. Fiorentini, and M. Lubasch, "Variational quantum amplitude estimation," *Quantum*, vol. 6, p. 670, 03 2022. [Online]. Available: https://doi.org/10.22331%2Fq-2022-03-17-670

[34] S. Chakrabarti, R. Krishnakumar, G. Mazzola, N. Stamatopoulos, S. Woerner, and W. J. Zeng, "A Threshold for Quantum Advantage in Derivative Pricing," *Quantum*, vol. 5, p. 463, 06 2021. [Online]. Available: https://doi.org/10.22331/q-2021-06-01-463

Appendix

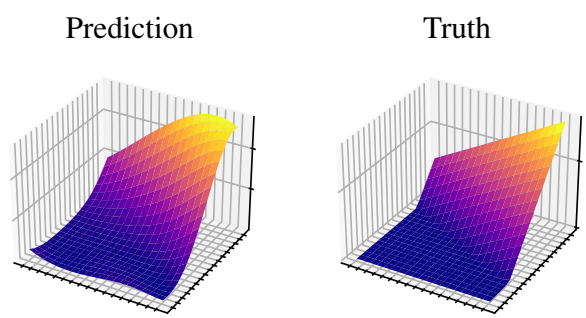


Fig. A.1: Payoff of a European basket call. On the left, the approximation by the circuit from Fig. A.3, on the right the true payout. On the X and Y axis are the prices of the two assets and on the Z axis the payoff.

Proof of Proposition 3.
For the sake of brevity, we write $|b(\boldsymbol{x})\rangle_c := \bigotimes_k |b(\boldsymbol{x}_k)\rangle_{r_k}$. First, we note that the assumption implies that U and $C_n(U)$ will generate the same quantum state when some data $\boldsymbol{x} \in \mathcal{X}$ is input,

$$(I_n \otimes U)\, |b(\boldsymbol{x})\rangle_c\, |0\rangle_t = C_n(U)\, |b(\boldsymbol{x})\rangle_c\, |0\rangle_t\,. \tag{7}$$

Then, for the right hand side of (6), it follows that

$$
\begin{aligned}
&f_{C_n(U),P}(\boldsymbol{\theta}) \\
&= \sum_{i,j=0}^{2^n-1} \sqrt{p_i}\sqrt{p_j} \left\langle b(x_i)\right|_c \left\langle 0\right|_t C_n(U)^\dagger \mathcal{M} C_n(U) \left|b(x_j)\right\rangle_c \left|0\right\rangle_t \\
&\overset{(7)}{=} \sum_{i,j=0}^{2^n-1} \sqrt{p_i}\sqrt{p_j} \left\langle b(x_i)\right|_c \left\langle 0\right|_t (I_n \otimes U)^\dagger \mathcal{M} (I_n \otimes U) \left|b(x_j)\right\rangle_c \left|0\right\rangle_t \\
&= \sum_{i=0}^{2^n-1} p_i \left\langle 0\right|_t U^\dagger M U \left|0\right\rangle_t \\
&= \sum_{i=0}^{2^n-1} p_i f_U(\boldsymbol{x}^i, \boldsymbol{\theta}).
\end{aligned}
$$

□

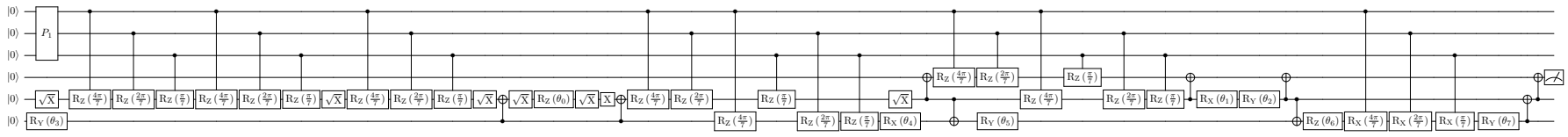


Fig. A.2: Controlled version of Fig. 4. A three qubit register is used as control.

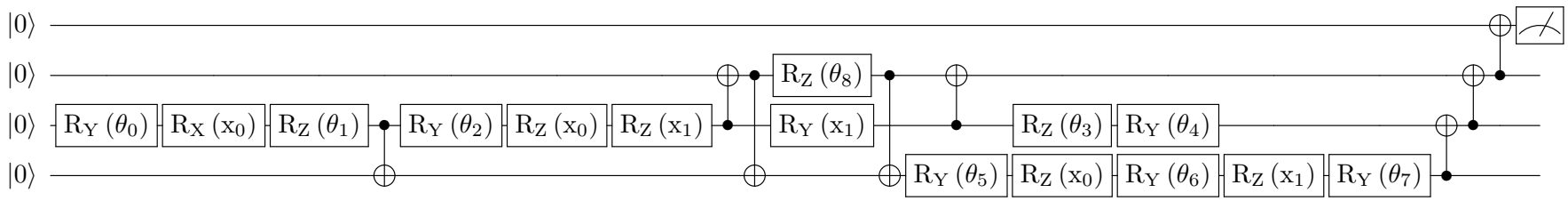


Fig. A.3: The genetically trained PQC for the payoff of a European basket option with fixed weight generated by Algorithm 1. $\boldsymbol{\theta}$ = [1.70789, -1.71191, -1.11194, -0.72190, -0.74404, 1.40678, 0.67897, 0.17417, -0.16803]

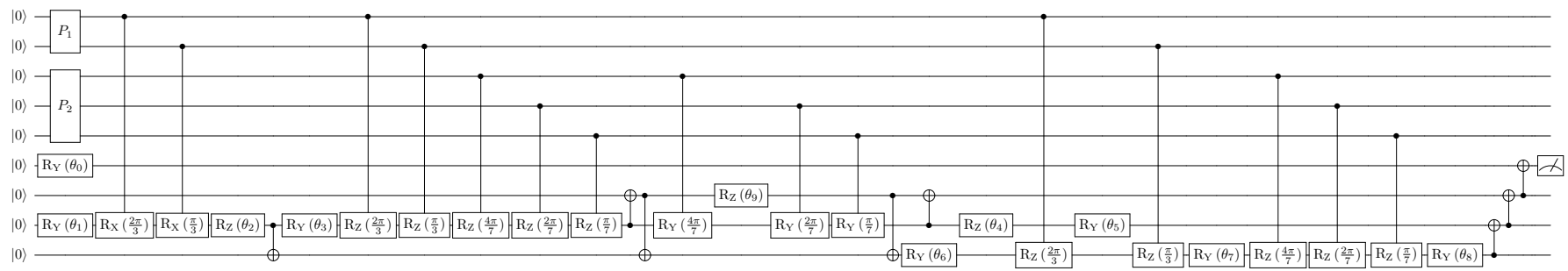


Fig. A.4: The CPQC constructed from the trained PQC Fig. A.3 combined with distribution loading operators P_1, P_2 for the European basket call with a fixed weight.

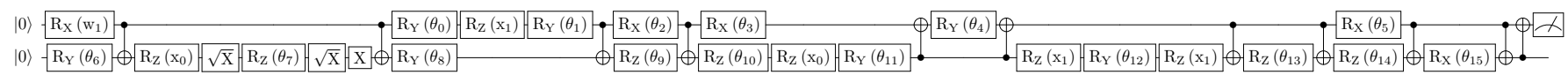


Fig. A.5: Genetically trained PQC for the payoff of a European basket call with variable weight generated by Algorithm 1. $\boldsymbol{\theta}$ = [1.34793, 1.39065, 0.56594, 0.40454, -2.45389, 1.54508, 2.17214, -0.50158, -0.63069, -1.14638, -0.44283, 0.44037, -0.42722, -0.20744, -0.23896, 0.10707]